

FinHive

De la teoria a la practica: una arquitectura multiagente jerarquica
financiera, construida sobre Databricks

Matias Fabian Adell

Agosto 2026

Resumen

Este articulo documenta el diseno, la implementacion y la puesta en produccion de FinHive: un sistema multiagente **jerarquico** de analisis financiero — un supervisor raiz que coordina cinco supervisores de dominio (macroeconomia, equity research, portfolio & risk, news & sentiment, y crypto/activos alternativos), cada uno con sus propios workers ReAct — construido sobre LangGraph y desplegado enteramente sobre Databricks Free Edition. El proyecto nace como aplicacion practica de una investigacion teorica previa sobre arquitecturas agenticas (timeline de 24 arquitecturas, 2020–2026) y busca demostrar, con un sistema real y verificado extremo a extremo, dominio de los patrones centrales del campo: ReAct, memoria persistente, RAG correctivo, topologias de orquestacion multiagente, protocolos de interoperabilidad, y gobernanza de modelos via un LLM Gateway. Mas alla de la arquitectura, este articulo documenta con el mismo nivel de detalle los **fallos reales encontrados y corregidos** durante la construccion — alucinaciones causadas por fallos silenciosos de herramientas, bugs de compatibilidad entre plataformas, errores de ruteo entre dominios — porque ese proceso de depuracion es, en la practica, donde mas se pone a prueba el conocimiento de como estos sistemas realmente fallan.

Índice

1. Introduccion	2
2. Contexto: de la teoria a un caso de uso real	2
3. Arquitectura del sistema	3
3.1. Vision general: supervisor de supervisores	3
3.2. Los cinco dominios	3
3.3. Infraestructura sobre Databricks	3
4. Mapa de conceptos teoricos aplicados	4
5. Decisiones de diseno y hallazgos tecnicos reales	5
5.1. El costo del LLM: tres decisiones antes de llegar a la correcta	5
5.2. Un bug de subprocess causa una alucinacion	6
5.3. Un supervisor que no sabia cuando parar	6
5.4. Ruteo incorrecto entre dominios y su costo	6
5.5. Tools defensivas: cuando una API externa falla	7
5.6. Model routing real, verificado empiricamente	7

6. Resultados	7
7. Trabajo futuro	7
8. Conclusion	8

1. Introduccion

Un modelo de lenguaje individual, por capaz que sea, tiene limites claros frente a un dominio tan amplio y heterogeneo como el analisis financiero: requiere datos macroeconomicos estructurados, fundamentals de empresas cotizantes, calculo cuantitativo de riesgo de portfolio, sentimiento de mercado en tiempo real, y datos de mercados de criptoactivos — cada uno con sus propias fuentes, formatos y ventanas temporales. La respuesta arquitectonica mas natural a esa heterogeneidad no es un unico agente generalista, sino una **red de agentes especializados**, coordinados bajo una topologia que distribuya la responsabilidad sin perder coherencia en la respuesta final.

FinHive implementa exactamente ese patron: una jerarquia de dos niveles donde un supervisor raiz decide, por cada turno de conversacion, a cual de cinco equipos de dominio delegar, y cada equipo de dominio es a su vez un supervisor que coordina entre dos y tres workers ReAct especializados. El proyecto se aloja integramente en **Databricks Free Edition**, aprovechando Unity Catalog, Foundation Model APIs nativos, Unity AI Gateway, y Databricks Repos — con el objetivo explicito de que el costo total de desarrollo y demostracion sea \$0.

Este articulo tiene tres objetivos. Primero, documentar la arquitectura y las decisiones de diseno que la sostienen. Segundo, mapear explicitamente que conceptos de la literatura de arquitecturas agenticas se aplican en que parte del sistema — no como referencia decorativa, sino como decisiones de diseno verificables. Tercero, y quizas mas valioso, documentar los **problemas reales** encontrados al construir un sistema de este tipo contra APIs y modelos reales, no simulados: qué falla, por qué falla, y cómo se corrigió.

2. Contexto: de la teoria a un caso de uso real

Este proyecto es la continuacion practica de un trabajo teorico previo — un timeline de 24 arquitecturas agenticas organizadas en seis eras narrativas, desde el RAG clasico ([1]) hasta los primeros intentos de sistematizar el campo en una taxonomia formal — que identifica una tendencia central: cada era del campo le devuelve control de diseno a un pipeline fijo y se lo entrega al propio sistema. Primero el modelo gana agencia sobre su propio control flow (ReAct, [2]); despues esa agencia se aplica al pipeline de RAG (Self-RAG, CRAG); despues se distribuye entre multiples agentes especializados bajo topologias de orquestacion (Network, Supervisor, Jerarquica); y finalmente se estandariza como esa agencia distribuida interopera entre organizaciones (MCP, A2A).

FinHive es, en ese sentido, un ejercicio deliberado de aplicar esa cadena completa a un dominio concreto en vez de quedarse en el nivel conceptual: la pregunta que organiza el proyecto no es “*que arquitectura elegir*” sino “*que tan poco control de diseno hace falta fijar a mano*” — topologia, memoria, tolerancia a fallos — delegando cada vez mas al propio sistema, en la misma direccion que senala la investigacion teorica previa.

3. Arquitectura del sistema

3.1. Vision general: supervisor de supervisores

La Figura 1 resume la topologia completa. El patron es el mismo, replicado en dos niveles: un nodo supervisor decide, con *structured output*, a que nodo delegar el siguiente turno, hasta decidir FINISH. En el nivel superior, los “trabajadores” son subgrafos completos (cada supervisor de dominio); en el nivel inferior, son workers ReAct individuales con tools propias.

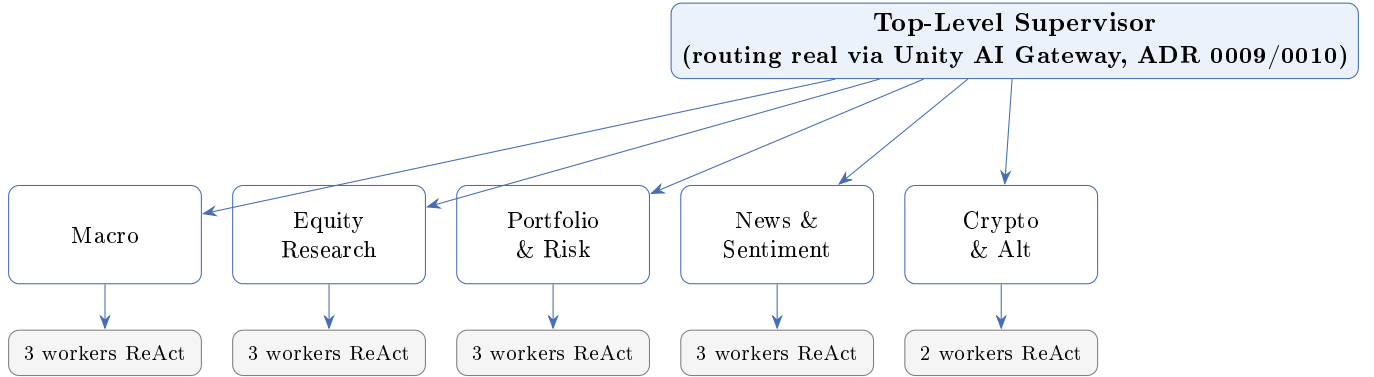


Figura 1: Topologia jerarquica de FinHive: un supervisor raiz enruta entre 5 equipos de dominio (patron “Supervisor”); cada equipo de dominio enruta entre sus propios workers ReAct (mismo patron, un nivel abajo). 13 workers en total sobre 25 tools registradas en Unity Catalog.

3.2. Los cinco dominios

- **Macro & Politica Monetaria:** tasas de interes, inflacion e indicadores generales, sobre datos reales de FRED (Federal Reserve Economic Data). Workers: `rates_worker`, `inflation_worker`, `indicators_worker`.
- **Equity Research & Fundamentals:** cotizaciones, fundamentals y filings regulatorios de empresas cotizantes, via yfinance y la API de SEC EDGAR (submissions y datos XBRL estructurados). Workers: `fundamentals_worker`, `technical_worker`, `filings_worker`.
- **Portfolio & Risk Management:** el unico dominio de *computo propio* en vez de passthrough a una API — volatilidad anualizada, VaR historico, matriz de correlacion y Sharpe ratio, calculados con `numpy/pandas` sobre precios reales. Workers: `allocation_worker`, `risk_worker`, `math_worker`.
- **News & Sentiment:** noticias con sentimiento estructurado y calendario de earnings via Alpha Vantage, con busqueda web (Tavily) como *fallback* estilo CRAG cuando la fuente estructurada no alcanza. Workers: `news_worker`, `sentiment_worker`, `calendar_worker`.
- **Crypto & Alternative Assets:** precio, historial y tendencias de criptomonedas via CoinGecko (API publica, sin autenticacion). Workers: `market_data_worker`, `alt_data_worker`.

3.3. Infraestructura sobre Databricks

Todo el sistema corre sobre **Databricks Free Edition**, con costo total verificado de \$0:

- **Foundation Model APIs nativos** (`system.ai.*`) para los LLMs de dominio: Llama 3.3 70B (supervisores) y Llama 3.1 8B (workers), sin ninguna key externa.
- **Unity Catalog Functions**: las 25 tools de datos estan registradas como funciones de Unity Catalog (`workspace.finhive.*`) para gobernanza y catalogacion — el equivalente conceptual a MCP sin pasar por los Managed MCP servers de Databricks, que facturan computo serverless por invocacion.
- **Unity AI Gateway**: el supervisor raiz — el nodo mas critico del grafo — corre sobre un *model service* propio (`workspace.finhive.finhive_router`) con **routing real** entre dos modelos (70 % Llama 3.3 70B / 30 % GPT OSS 120B), mas rate limits explicitos sobre los tres endpoints del proyecto.
- **Databricks Repos + notebooks nativos**: el repositorio de GitHub esta conectado como Repo en el workspace; un notebook demo (`00_demo.py`) instala el paquete en modo editable y corre el sistema completo dentro de Databricks, con credenciales cargadas via `dbutils.secrets` y `dbutils.notebook` en vez de archivos de configuracion locales.
- **MLflow Tracing**: cada invocacion del grafo genera una traza completa (cada nodo, cada tool call, cada llamada a LLM) via `mlflow.langchain.autolog()`.

4. Mapa de conceptos teoricos aplicados

La Tabla 1 conecta explicitamente cada concepto de la investigacion teorica previa con su implementacion concreta en FinHive — el objetivo declarado del proyecto no es solo construir un sistema que funcione, sino demostrar, de forma verificable, el manejo de estos patrones.

Cuadro 1: Mapa de trazabilidad teoria–implementacion.

Concepto (pers/seccion)	(pa-	Donde se aplica en FinHive
ReAct [2]	Los	13 workers hoja usan <code>langchain.agents.create_agent</code> (patron thought-action-observation).
Reflexion [3]		Prompt de auto-critica en los supervisores de dominio: no aceptar un dato sin que provenga de una tool real.
Generative Agents / MemGPT [4, 5]		Memoria de dos niveles planificada (checkpointing de conversacion + memoria persistente de perfil), trabajo futuro documentado en la seccion 7.
Self-RAG / Corrective RAG [6, 7]		<code>web_search_news</code> (Tavily) como fallback cuando Alpha Vantage no cubre una consulta — patron CRAG de “no confiar ciegamente en la fuente primaria”.
Multi-Agent Network / Supervisor / Jerarquica [9, 10]		Los tres patrones del proyecto de referencia, aplicados literalmente: cada dominio es un “Supervisor” de sus workers; el conjunto completo es “Jerarquico” (TalkHier) de dos niveles.

Cuadro 1: Mapa de trazabilidad teoria–implementacion.

Concepto (pers/seccion)	(pa-	Donde se aplica en FinHive
Mixture-of-Agents [11]		Sintesis del supervisor raiz cuando una pregunta cruza dos o mas dominios, combinando las respuestas de equipos independientes en una unica respuesta coherente.
Model Context Protocol (MCP)		Unity Catalog Functions expuestas con interfaz gobernada y catalogada, en vez de los Managed MCP servers (decision de costo, ver ADR 0004).
LLM Gateway (§19, resumen conceptual del boot-camp)		Unity AI Gateway: routing real 70/30 entre dos modelos, rate limits explicitos, usage tracking — las mismas responsabilidades (routing/fallback, cost tracking, policy enforcement) descriptas en la literatura, verificadas en un sistema real.
Adaptive-RAG [8]		El router del supervisor raiz decide, por turno, si delegar a un unico equipo o a varios — version simplificada de enrutar segun la complejidad real de la pregunta.
LLM-as-judge / evaluacion		Trabajo futuro directo: Mosaic AI Agent Evaluation (framework CLEARs) sobre el grafo ya instrumentado con MLflow.

5. Decisiones de diseno y hallazgos tecnicos reales

Esta seccion documenta, en orden cronologico, los problemas reales encontrados construyendo el sistema contra APIs y modelos reales — no como anecdotas, sino porque en la practica es donde mas se pone a prueba la comprension de como fallan estos sistemas. El registro completo esta en diez *Architecture Decision Records* (ADRs) en el repositorio; esta seccion resume los mas significativos.

5.1. El costo del LLM: tres decisiones antes de llegar a la correcta

La primera decision de arquitectura — que LLM usar — paso por tres iteraciones reales antes de asentarse. Primero se evaluo Anthropic via *External Models* de Databricks; al confirmar que la API de Anthropic es pay-per-uso separada de una suscripcion de Claude, se paso a Groq (free tier real, modelos open-weight). Groq quedo configurado y probado, pero al revisar el catalogo de Foundation Model APIs *nativos* de Databricks se confirmo, con una llamada real contra el workspace, que estan incluidos sin costo en Free Edition — sin necesitar ninguna cuenta de terceros. Se migro la arquitectura completa a los modelos nativos, eliminando toda dependencia de key externa para el LLM.

Hallazgo: decisiones de proveedor de LLM no son solo tecnicas

El costo real de un proveedor de LLM no siempre es evidente hasta probarlo contra la cuenta real: la documentacion generica no sustituye una llamada de prueba contra el propio workspace.

Las tres iteraciones quedaron documentadas en ADRs sucesivos en vez de sobrescribir la decision anterior — el registro del cambio es en si mismo parte del valor del documento.

5.2. Un bug de subprocess causa una alucinacion

Al conectar las tools de datos como *Unity Catalog Functions*, el modo recomendado de ejecucion (`UCFunctionToolkit` en modo `local`) fallo silenciosamente en Windows: el runner que genera internamente hace `import resource`, un modulo de la libreria estandar que **solo existe en sistemas Unix**. El fallo se manifesto como un mensaje de error generico (“*the function execution has been terminated with a signal*”), y el worker, en vez de admitir que no tenia el dato, **alucino una cifra plausible** (5.25 % de tasa de fondos federales, cuando el valor real — verificado por separado — era 3.63 %).

Hallazgo: un fallo silencioso de tool es indistinguible de un dato real

La correccion tecnica fue simple (envolver las mismas funciones directamente como tools de LangChain, sin pasar por el sandbox de subprocess), pero el hallazgo de fondo es mas general: cuando una tool falla sin una senal clara, el modelo de lenguaje puede optar por una respuesta plausible en vez de una admision de incertidumbre. Ese comportamiento es exactamente lo que Corrective RAG y los guardrails de *groundedness* buscan prevenir — y esta fue evidencia empirica de por que importan, no solo una referencia teorica.

5.3. Un supervisor que no sabia cuando parar

Al implementar el supervisor raiz con *structured output* para decidir el proximo equipo, dos problemas de compatibilidad y de disenio de prompt aparecieron en la misma iteracion. Primero, un `Literal` construido dinamicamente con *unpacking* en runtime rompio la conversion de schema de `with_structured_output` (`TypeError: issubclass() arg 1 must be a class`) — resuelto usando un campo `str` simple, validado en codigo en vez de delegarlo al schema. Segundo, y mas relevante: sobre una pregunta ya respondida completamente en la primera vuelta, el supervisor re-ruteo al mismo equipo **nueve veces** antes de decidir `FINISH`, gastando cuota de requests sin necesidad.

La correccion combino dos capas, deliberadamente redundantes: un prompt mas explicito sobre cuando parar, y un **limite duro de iteraciones** independiente del juicio del modelo. Con ambas, la misma clase de pregunta paso de nueve vueltas a una.

5.4. Ruteo incorrecto entre dominios y su costo

Con cuatro dominios reales, la pregunta “*¿cuando es el proximo earnings de Apple?*” se enruto al equipo de **Equity** en vez de **News & Sentiment**. El router solo recibia los *nombres* de los equipos, sin descripcion de sus limites — nada distinguia “resultados ya reportados” de “fecha de un resultado futuro”. Equity, sin la tool de calendario, **alucino una fecha incorrecta** (2025 en vez de la real, octubre de 2026) en vez de admitir que no tenia el dato: el mismo patron de fondo que el hallazgo de la seccion 5.2, esta vez causado por una ambigüedad de ruteo en vez de un bug de plataforma.

La correccion fue agregarle al router descripciones explicitas de cada equipo, incluyendo lineas de “esto NO es de este equipo” en los casos de frontera ya identificados — un patron que se aplico preventivamente al sumar el quinto dominio (Crypto), anotando de antemano su frontera con Equity antes de que apareciera el mismo tipo de bug.

5.5. Tools defensivas: cuando una API externa falla

Probando el dominio de Crypto bajo uso intensivo, un **429 Too Many Requests** de CoinGecko sin capturar **crasheo la invocacion completa** del grafo jerarquico de cinco equipos — la excepcion se propagaba en vez de quedar como una observacion que el worker pudiera manejar. El problema era transversal, no especifico de un dominio: los cinco usan el mismo patron `requests.get(...); response.raise_for_status()`. La correccion fue un decorador (`safe_tool`) que envuelve cualquier tool y convierte cualquier excepcion en un string de error descriptivo, aplicado en las 25 tools de los cinco dominios.

5.6. Model routing real, verificado empiricamente

El hallazgo mas reciente no fue un bug sino una capacidad nueva: Unity AI Gateway (disponibilidad general, agosto de 2026) permite crear *model services* de Unity Catalog con routing real entre modelos. Se configuro `finhive_router` con 70 % de trafico a Llama 3.3 70B y 30 % a GPT OSS 120B, y se **verifico empiricamente**, no solo por configuracion: seis llamadas de prueba devolvieron cinco respuestas servidas por Llama y una por GPT OSS 120B, consistente con el split configurado. El cliente de LangChain especializado de Databricks (`ChatDatabricks`) todavia no soporta el path nuevo de API que exige este mecanismo; la integracion se resolvio usando el cliente OpenAI-compatible que Databricks expone directamente (`langchain_openai.ChatOpenAI`, apuntando a `/ai-gateway/mlflow/v1`), sin esperar a que la libreria especializada alcance a la plataforma.

6. Resultados

- Cinco dominios completos, verificados extremo a extremo contra Databricks y APIs de datos reales — no simulados.
- 13 workers ReAct sobre 25 Unity Catalog Functions registradas y gobernadas.
- Seis tests de integracion pasando juntos, cada uno verificando que la respuesta este *grounded* en una llamada real a una tool (no solo que el codigo no arroje una excepcion).
- Costo total de LLM y embeddings: **\$0**, verificado en vivo contra la cuenta real de Databricks Free Edition, no solo asumido por documentacion.
- Diez Architecture Decision Records documentando cada decision y cada hallazgo con su razonamiento, no solo el estado final.

7. Trabajo futuro

- **Guardrails formales**: tópico, seguridad, *groundedness* y anti-jailbreak, con disclaimers explicitos — motivado directamente por los hallazgos de alucinacion documentados en este articulo.

- **Memoria persistente:** checkpointer de conversacion mas memoria de largo plazo de perfil/portfolio (patron MemGPT), pendiente desde el diseno original.
- **RAG sobre filings completos:** arbol de resúmenes al estilo RAPTOR sobre el texto completo de 10-K/10-Q, hoy cubierto solo por metadata y datos XBRL estructurados.
- **Mosaic AI Agent Framework:** envolver el grafo existente (sin reescribirlo) con la interfaz ChatAgent de MLflow y evaluar con el framework CLEARS (Correctness, Latency, Execution, Adherence, Relevance, Safety) — extension directa de la instrumentacion de MLflow ya activa.
- **Demo desplegada:** aplicacion Streamlit como Databricks App, con visualizacion de que equipo/worker respondio cada parte de la pregunta.

8. Conclusion

FinHive demuestra que los patrones centrales de la literatura de arquitecturas agenticas — ReAct, RAG correctivo, topologias de orquestacion jerarquica, protocolos de gobernanza de herramientas y modelos — no son solo conceptos de paper, sino decisiones de ingenieria con consecuencias concretas y verificables cuando se aplican contra sistemas reales. El valor mas duradero del proyecto no es la arquitectura final en si misma, sino el registro explicito de cada decision y cada fallo real que llevo hasta ahi: una alucinacion causada por un bug de plataforma, un loop sin limite, un error de ruteo entre dominios — cada uno documentado con su causa raiz y su correccion, no solo su resultado. Es esa disciplina de documentacion, mas que cualquier eleccion de framework, la que hace que el sistema sea auditable y extensible.

Referencias

- [1] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 33*, 9459–9474.
- [2] Yao, S. et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*.
- [3] Shinn, N. et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS*.
- [4] Park, J. S. et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. *UIST*.
- [5] Packer, C. et al. (2023). MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*.
- [6] Asai, A. et al. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *ICLR*.
- [7] Yan, S.-Q. et al. (2024). Corrective Retrieval Augmented Generation. *arXiv:2401.15884*.
- [8] Jeong, S. et al. (2024). Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. *NAACL*.
- [9] Wu, Q. et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv:2308.08155*.

- [10] Wang, Z. et al. (2025). Talk Structurally, Act Hierarchically: A Collaborative Framework for LLM Multi-Agent Systems. *arXiv preprint*.
- [11] Wang, J. et al. (2024). Mixture-of-Agents Enhances Large Language Model Capabilities. *arXiv:2406.04692*.
- [12] Adell, M. (2026). Del bibliotecario solitario al equipo de agentes: una linea de tiempo de las arquitecturas agenticas, de RAG a los sistemas multiagente (2020–2026). Seminario de Tendencias de Ciencia de Datos.